

Experiment No 01

AIM:- Cryptography in Blockchain, Merkle root Tree Hash

Collab Link:- [Experiment](#)

THEORY

1. Cryptographic Hash Functions in Blockchain

Cryptographic hash functions are mathematical algorithms that take an input (or "message") and return a fixed-size string of bytes. The output, typically a digest, is unique to each specific input. In blockchain, these functions are essential for ensuring data integrity and security. Key properties include being deterministic (same input always produces same output), quick to compute, pre-image resistant (hard to reverse), and collision-resistant (two different inputs shouldn't produce the same hash).

2. What is a Merkle Tree?

A Merkle Tree, also known as a binary hash tree, is a data structure used in computer science and cryptography to efficiently summarize and verify the integrity of large sets of data. In a blockchain, it is used to store all the transactions in a block by hashing pairs of transactions until only one hash remains: the Merkle Root.

3. Cryptographic Puzzle and the Golden Nonce

A Cryptographic Puzzle is a computationally difficult problem that miners must solve to add a new block to the blockchain, often referred to as Proof of Work. The "Golden Nonce" is the specific nonce (number used once) value that, when hashed with the rest of the block's data, produces a hash that meets the network's difficulty target (e.g., a hash starting with a certain number of zeros).

4. How does a Merkle Tree work?

It works by organized bottom-up hashing. First, every transaction at the bottom level (leaf nodes) is hashed. Then, pairs of these hashes are concatenated and hashed again to form the level above. This process repeats until a single hash, the Merkle Root, is formed at the top. If there is an odd number of transactions, the last one is duplicated to maintain the binary structure.

5. Benefits of Merkle Tree

The primary benefits include efficient verification of data (you don't need to download the whole block to verify a transaction), reduced disk space and bandwidth usage, and high security. It allows for "Simplified Payment Verification" (SPV), where a user can verify that a transaction is included in a block without needing to process the entire blockchain.

6. Use Cases of Merkle Tree

Beyond Bitcoin and Ethereum for transaction verification, Merkle Trees are used in distributed systems like Apache Cassandra and Riak to detect inconsistencies between data replicas. They are also used in Git for version control to identify file changes and in Certificate Transparency logs to prove that a website certificate has been publicly logged.

Program 1

```
import hashlib #error state

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest() #error sta
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result) #error st

Enter a string: Ved
Hash: 53aebfb97c640b6a1b0135f9ff445ba1e858564848db46f6fb7bb99610280167
```

Program 2

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce # Error

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code) # Err

Enter a string: Ved Joshi
Enter the nonce: 204121
Hash Code: 000004d02b85146f9e796ab18a897c8b440382395c84d7f8407beff76043272c
```

Program 3

```

import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
            return nonce

        nonce += 1

# Get user input
input_string = "Ved Joshi"
num_zeros = 6

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)

```

```

Hash: 00000064f4345cc767f976e303d5c192ff9128791d8b64796b173fec3ad7b298
Input String: Ved Joshi
Leading Zeros: 6
Expected Nonce: 10105765

```

Program 4

```

import hashlib

def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None

    # Hash each initial transaction
    hashed_transactions = [hashlib.sha256(t.encode('utf-8')).hexdigest() for t in transactions]
    print(f"Initial Hashed Transactions: {hashed_transactions}")

    # If only one transaction, its hash is the root
    if len(hashed_transactions) == 1:
        return hashed_transactions[0]

    # Recursive construction of the Merkle Tree
    current_level_hashes = hashed_transactions
    level = 1

    while len(current_level_hashes) > 1:
        print(f"\nBuilding Merkle Tree - Level {level}:")

        # If odd number of hashes, duplicate the last one
        if len(current_level_hashes) % 2 != 0:
            current_level_hashes.append(current_level_hashes[-1])

        next_level_hashes = []
        for i in range(0, len(current_level_hashes), 2):
            combined = current_level_hashes[i] + current_level_hashes[i+1]
            hash_combined = hashlib.sha256(combined.encode('utf-8')).hexdigest()
            next_level_hashes.append(hash_combined)

        print(f"    Combining {current_level_hashes[i]} and {current_level_hashes[i+1]} -> {hash_combined}")

        current_level_hashes = next_level_hashes
        level += 1

    return current_level_hashes[0]

# Updated Example usage
transactions = [
    'Alice \u2192 Bob : $200',
    'Bob \u2192 Dave : $500',
    'Dave \u2192 Eve : $100',
    'Eve \u2192 Alice : $300',
    'Roo \u2192 Bob : $50'
]

merkle_root = build_merkle_tree(transactions)
print(f"\nFinal Merkle Root: {merkle_root}")

```

Initial Hashed Transactions:

```

['d7d1c2448f818062ab9bb89aa9f95564670895f927d730d2d43ba8e5cada8a4d',
'ebc396058242fc4cb711aa1dd2c2ce1c44818b9d35c9d53465bee3c12462072e',
'56a42d8ba343cbcf8133e8a0300564876f2e76dfe8281db52fccbad24fc8d88a',
'fc3d2297f8b8aa145fa4ded66a1ed06db9d46d661a8d3673e1ceb0dbbaf597a6',
'70ec9a2f7f21cf62ef7acfd60032bcc094a2739d2ae73dea3e929512b9a00c4a']

```

Building Merkle Tree - Level 1:

Combining d7d1c2448f818062ab9bb89aa9f95564670895f927d730d2d43ba8e5cada8a4d and
ebc396058242fc4cb711aa1dd2c2ce1c44818b9d35c9d53465bee3c12462072e ->
51215ac8c39b7d8d72ce873c5cf75a9b7d3edd069c79007a1f0827af3e43d258

Combining 56a42d8ba343cbcf8133e8a0300564876f2e76dfe8281db52fccbad24fc8d88a and
fc3d2297f8b8aa145fa4ded66a1ed06db9d46d661a8d3673e1ceb0dbbaf597a6 ->
b5e0a195812483b212784056d2bf15f8caeff4fb4bde6e4910d9827d039632f1

Combining 70ec9a2f7f21cf62ef7acfd60032bcc094a2739d2ae73dea3e929512b9a00c4a and 70ec9a2f7f21cf62ef7acfd60032bcc094a2739d2ae73dea3e929512b9a00c4a -> 7b5c2d1c9a72494009e89c8608db836a79e71e13f5d7f12822b026364a22b5b6

Building Merkle Tree - Level 2:

Combining 51215ac8c39b7d8d72ce873c5cf75a9b7d3edd069c79007a1f0827af3e43d258 and b5e0a195812483b212784056d2bf15f8caeff4fb4bde6e4910d9827d039632f1 -> 8e3aa833a93cf5663d5138a8d80d8e6a8cf006517a462fb4260806795701f45d

Combining 7b5c2d1c9a72494009e89c8608db836a79e71e13f5d7f12822b026364a22b5b6 and 7b5c2d1c9a72494009e89c8608db836a79e71e13f5d7f12822b026364a22b5b6 -> 5b0369cfb2de9ed3b41d380675c240323de05bc9b9062fae8cd5991902828cd2

Building Merkle Tree - Level 3:

Combining 8e3aa833a93cf5663d5138a8d80d8e6a8cf006517a462fb4260806795701f45d and 5b0369cfb2de9ed3b41d380675c240323de05bc9b9062fae8cd5991902828cd2 -> 382f188ce8666903c084f5704ade4a7e1a1948242bdfd0de2b529d9fedb6fe2f

Conclusion

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing provides a secure and unique fingerprint for any input, ensuring data integrity.**
- **Nonce and Target Hashing allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).**
- **Proof-of-Work Puzzle Solving shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.**
- **Merkle Tree Construction efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.**

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.